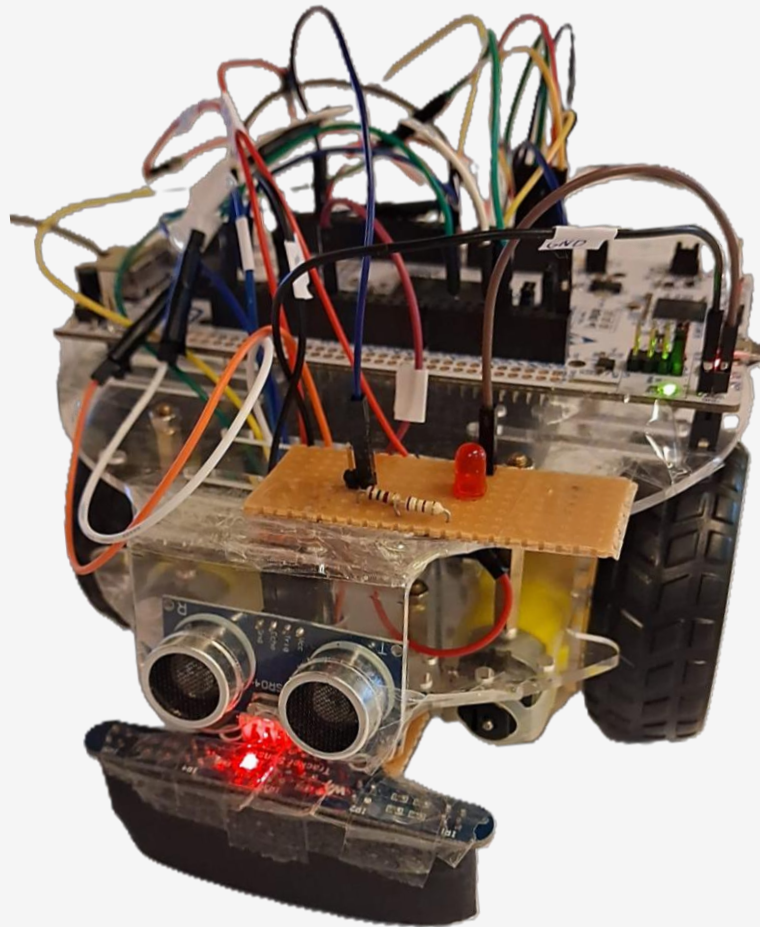


Relatório Final de Sistemas Embebidos

Robô seguidor de linha com detecção de obstáculos



Grupo 4

Catarina Sousa – a97259

Isabel Gomes – a104868



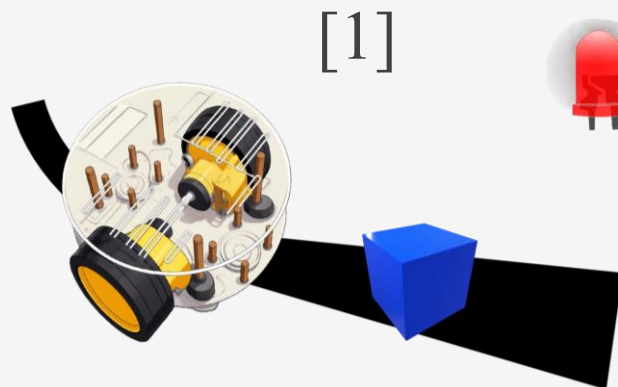
UC: Sistemas Embebidos
Ano Letivo: 2025/2026 – 2º Semestre

1. Definição do problema

Consiste num robô autónomo capaz de seguir uma linha, detetar um obstáculo e reagir perante esse evento.

O robô segue a linha normalmente e, quando encontra um obstáculo para e sinaliza através do acionamento de um LED vermelho[1].

Assim que o obstáculo desaparece, o LED é desativado e o robô retoma o percurso[2].



1. Definição do problema

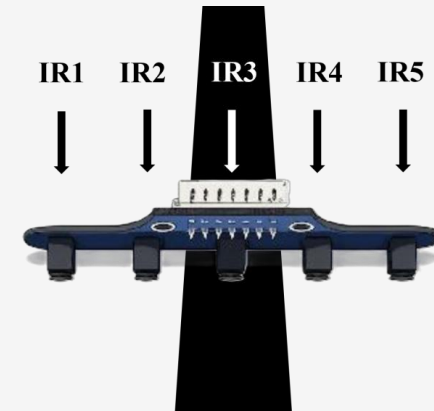
O robô é capaz de:

- Seguir uma linha através de sensores infravermelhos;
- Detetar obstáculos através de um sensor ultrassónico;
- Parar os motores quando deteta um obstáculo;
- Sinalizar esse evento através do acionamento de um LED;
- Retomar o seguimento da linha quando o obstáculo desaparece.

1. Definição do problema

Componentes utilizados:

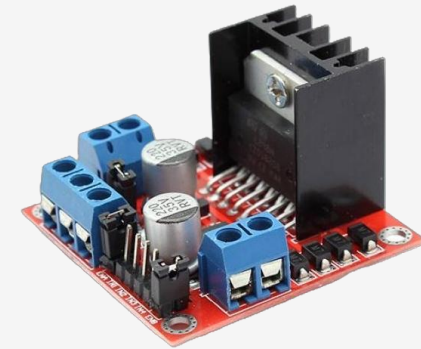
- Para o correto seguimento do percurso, utilizou-se um **módulo de 5 sensores IR**.
- Na deteção de obstáculos, o **sensor ultrassónico HC- SR04P**.



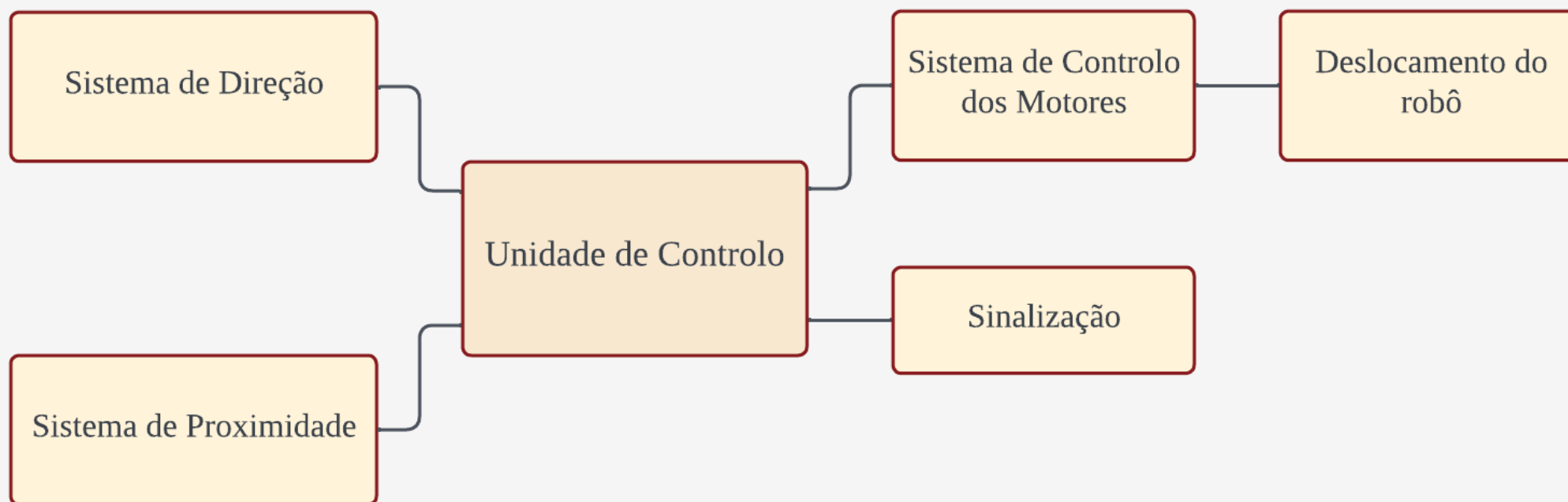
1. Definição do problema

Componentes utilizados:

- Para o controlo independente dos motores DC, um **módulo driver motores L298N**.
- Como sinalização, um **LED vermelho**.
- Dois **motores DC (3V-6V)**.



2. Diagrama de Blocos



3. Design do sistema

3.1. Algoritmo geral - Pseudocódigo

Início do programa

Definir distância limite para detecção de obstáculo

Definir velocidade base de cada motor

Enquanto o robô estiver ligado:

 Ler sensor ultrassônico

 Calcular distância ao obstáculo

 Ler os 5 sensores IR

 Se a distância for menor que o limite definido:

 Parar os motores

 Ligar LED vermelho

 Senão:

 Desligar o LED vermelho

Se o sensor central detetar linha:

 Avançar em frente

 Motor esquerdo = velocidade base

 Motor direito = velocidade base

Senão se os sensores da esquerda detetam linha:

 Corrigir para a esquerda

 Motor esquerdo = velocidade reduzida

 Motor direito = velocidade maior

Senão se os sensores da direita detetam linha:

 Corrigir para a direita

 Motor esquerdo = velocidade maior

 Motor direito = velocidade reduzida

Senão:

 Parar os motores

Fim do ciclo

3. Design do sistema

3.2. Instanciação do sistema nas plataformas computacionais

Sistema de Direção:

Implementado pelo módulo de 5 sensores IR, que tem como função detetar a posição da linha relativamente ao robô. Os sinais analógicos dos sensores são lidos pela STM32 através do ADC, e convertidos para valores digitais para processamento.

Módulo de 5 Sensores IR		
IR1	Pino PA0	ADC1_INP16 (<i>single ended</i>)
IR2	Pino PA3	ADC1_INP15 (<i>single ended</i>)
IR3	Pino PA4	ADC1_INP18 (<i>single ended</i>)
IR4	Pino PA5	ADC1_INP19 (<i>single ended</i>)
IR5	Pino PA6	ADC1_INP3 (<i>single ended</i>)
VCC	Pino 3,3V	-
GND	Pino GND	-

3. Design do sistema

3.2. Instanciação do sistema nas plataformas computacionais

Sistema de Proximidade:

Implementado pelo sensor ultrassónico HC-SR04P, que mede a distância a que se encontram os objetos. Tem como saída a distância calculada pela STM32.

Sensor Ultrassónico		
VCC	Pino 3,3V	-
TRIG	Pino PB2	GPIO OUTPUT
ECHO	Pino PB10	GPIO INPUT (<i>pull-down</i>)
GND	Pino GND	-

3. Design do sistema

3.2. Instanciação do sistema nas plataformas computacionais

Unidade de controlo:

Implementada pela STM32 NUCLEO-H755ZI-Q.

A instanciação na STM32 foi realizada através da utilização dos periféricos ADC, GPIO e TIM, permitindo-lhe realizar diversas funções, entre as quais:

- leitura dos sensores IR;
- leitura do sensor ultrassónico;
- execução do algoritmo de decisão;
- geração dos sinais PWM;
- controlo do LED.

3. Design do sistema

3.2. Instanciação do sistema nas plataformas computacionais

Sistema de controlo dos motores:

Implementado pelo módulo driver motores L298N. Tem como funções, receber sinais PWM gerados pela STM, regular a potência aplicada aos motores e controlar os motores de forma independente.

Módulo Driver Motores L298N		
ENA	Pino PC6	TIM3_CH1 (PWM Generation)
ENB	Pino PC7	TIM3_CH2 (PWM Generation)
IN1	Pino PC8	GPIO OUTPUT
IN2	Pino PC9	GPIO OUTPUT
IN3	Pino PC10	GPIO OUTPUT
IN4	Pino PC11	GPIO OUTPUT
12V	Alimentação+	-
GND	Pino GND, Alimentação-	-
OUT1	Motor1-	-
OUT2	Motor1+	-
OUT3	Motor2+	-
OUT4	Motor2-	-

3. Design do sistema

3.2. Instanciação do sistema nas plataformas computacionais

Deslocamento do robô:

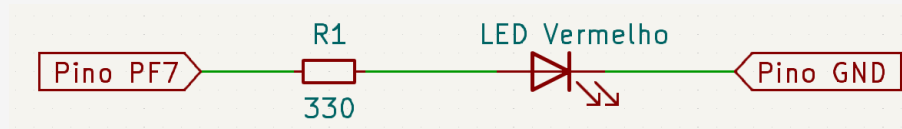
Implementado por dois motores DC (3V-6V), que produzem o movimento físico do robô.

Sinalização:

Implementado por um LED vermelho, indicando a presença do obstáculo.

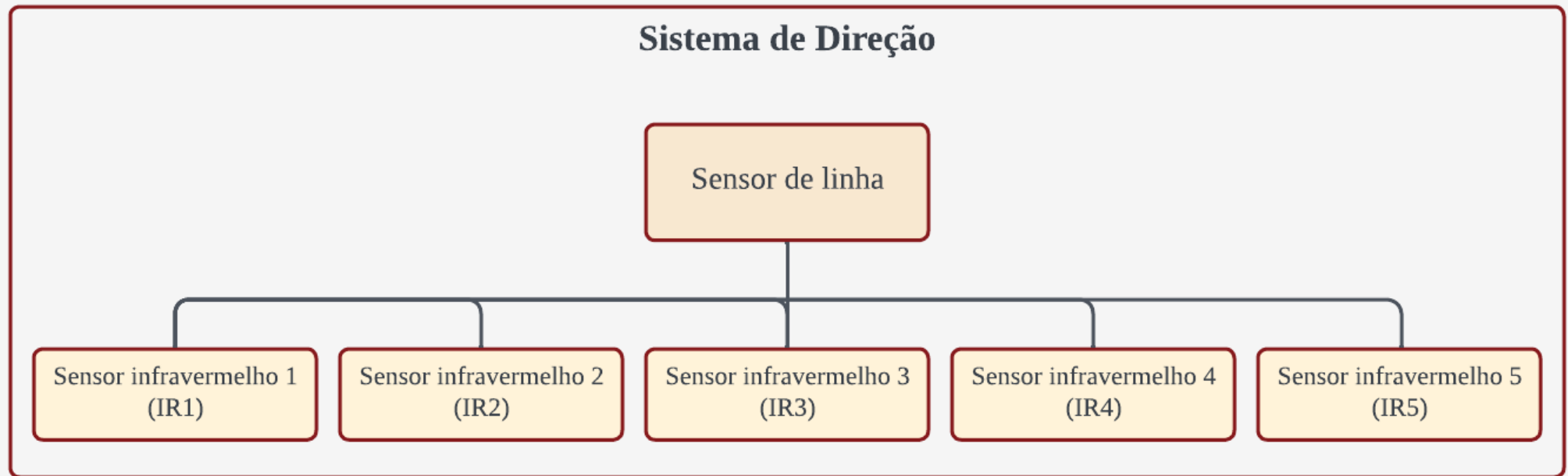
LED Vermelho		
Ânodo	Pino PF7	GPIO OUTPUT (<i>push-pull</i>)
Cátodo	Pino GND	-

Esquemático:



4. Design detalhado dos blocos do diagrama

Sistema de Direção:



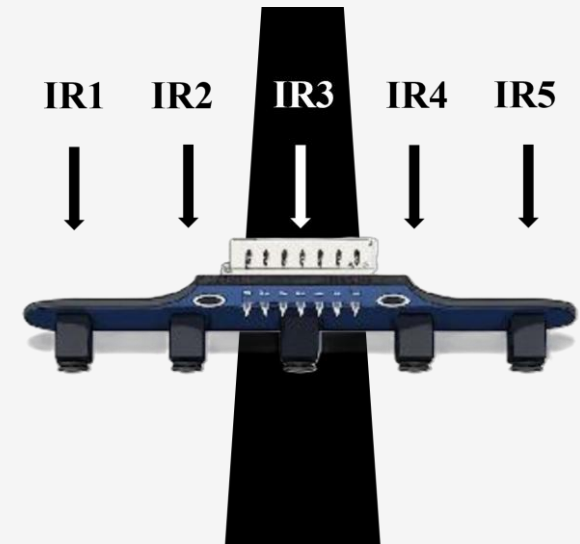
4. Design detalhado dos blocos do diagrama

Sistema de Direção:

Um módulo com 5 sensores infravermelhos, permite detetar a linha a seguir. Cada sensor emite luz IR e mede a luz refletida pela superfície. Zonas claras refletem mais luz, enquanto a linha escura reflete menos.

Com base nos valores lidos pelos sensores, a STM32 consegue perceber se o robô está centrado na linha, deslocado para a esquerda ou para a direita.

Assim, o controlador ajusta a velocidade dos motores para corrigir a direção do robô.



4. Design detalhado dos blocos do diagrama

Sistema de Direção:

Valores práticos obtidos:

Perante a cor branca					
Sensores	IR1	IR2	IR3	IR4	IR5
Analógico	3,09 V	3,13 V	3,12 V	3,10 V	3,09 V
Digital	3845	3890	3882	3847	3838
Perante a cor preta					
Analógico	1,22 V	0,55 V	0,98 V	0,97 V	1,12 V
Digital	1524	680	1220	1208	1384

A gama de tensão varia entre 0 V e 3,3 V, correspondente à faixa de operação do conversor analógico-digital (ADC) da STM32.

Já os valores digitais resultantes da conversão encontram-se compreendidos entre 0 e 4095, dado que o ADC utilizado possui uma **resolução de 12 bits** ($2^{12} = 4096$ níveis possíveis).

Os valores obtidos estão de acordo com a operação matemática

típica da conversão ADC: $Valor\ Digital = \frac{Tensão\ aplicada}{3,3V} * 4096$

4. Design detalhado dos blocos do diagrama

Sistema de Direção:

Módulo de software: sensor_linha.c / sensor_linha.h

Interface:

```
void LineSensor_Read(void);  
void LineSensor_ConvertToDigital(void);  
uint16_t* LineSensor_GetAnalogValues(void);  
uint8_t* LineSensor_GetDigitalValues(void);
```


4. Design detalhado dos blocos do diagrama

Sistema de Direção:

Algoritmo das funções:

LineSensor_Read()

- Lê os 5 sensores infravermelhos através do ADC1;
- Cada sensor é lido num canal ADC diferente;
- Os valores obtidos são guardados no vetor ir_values[].

LineSensor_ConvertToDigital()

- Compara cada valor analógico com o limite definido;
- Se o valor lido for inferior ao limite, considera que a linha preta foi detetada;
- O resultado é guardado no vetor line[], com valores 0 ou 1.

4. Design detalhado dos blocos do diagrama

Sistema de Direção:

Algoritmo das funções:

LineSensor_GetAnalogValues()

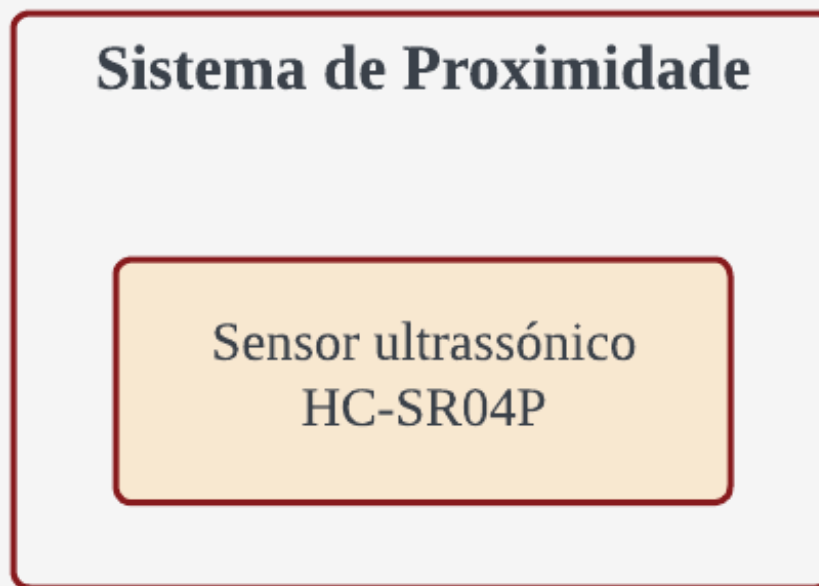
- Devolve o vetor com valores analógicos lidos dos sensores IR;
- Esta função é usada para debug e calibração dos limites;

LineSensor_GetDigitalValues()

- Devolve o vetor digital line[];
- Este vetor é usado pela main.c para decidir se o robô deve avançar, corrigir para a esquerda, corrigir para a direita, ou parar.

4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:



4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:

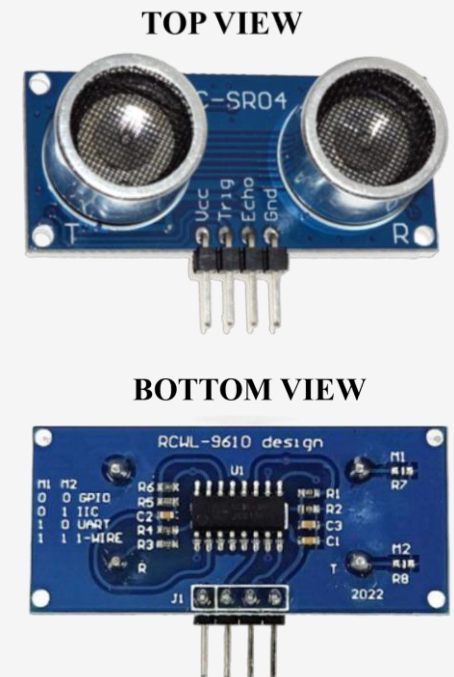
O sensor ultrassônico HC-SR04P permite detectar a presença de obstáculos à frente do robô.

Este sensor funciona enviando um pulso sonoro que reflete no obstáculo e regressa ao sensor sob a forma de eco.

A STM32 mede o tempo entre o envio do sinal e a recepção.

Assim, é possível calcular a distância ao obstáculo.

Se essa distância for inferior a um limite definido, o sistema considera que existe um obstáculo.



4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:

Módulo de software: sensor_ultrassonico.c / sensor_ultrassonico.h

Interface:

```
#define OBSTACLE_DISTANCE_CM 15.0f  
void Ultrasonic_Init(void);  
float Ultrasonic_GetDistance_cm(void);
```

4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:

Algoritmo das funções:

Ultrasonic_Init()

- Inicializa o sensor ultrassónico através do arranque do temporizador TIM2;
- Este temporizador é utilizado para gerar pequenos atrasos em microssegundos e medir a duração do sinal ECHO.

Ultrasonic_GetDistance_cm()

- Coloca o pino TRIG a nível baixo durante 2us;
- Gera um pulso de 10us no pino TRIG;
- Aguarda que o pino ECHO passe para nível alto.

4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:

Algoritmo das funções:

- Reinicia o contador do TIM2;
- Mede o tempo durante o qual o pino ECHO permanece a nível alto;
- Se o tempo de espera ultrapassar o limite definido, retorna um valor de erro;
- Converte o tempo medido para a distância em centímetros , através da relação:

$$\text{distância} = \text{tempo} / 58;$$

- Retorna a distância calculada.

4. Design detalhado dos blocos do diagrama

Sistema de Proximidade:

Função auxiliar interna:

delay_us(uint16_t* us)

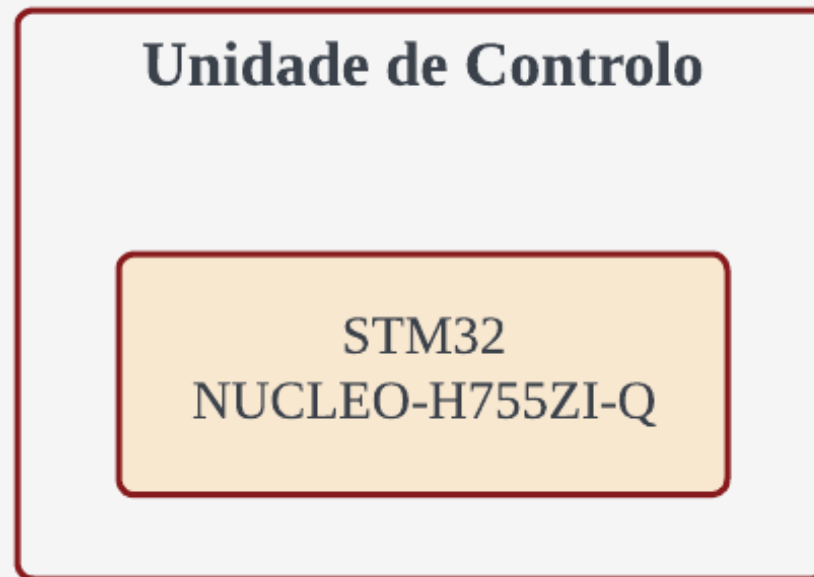
- Gera um atraso em microssegundos utilizando o contador TIM2;
- Esta função é interna ao módulo, e é usada para criar o pulso de trigger do sensor.

Utilização na main:

```
float distance = Ultrasonic_GetDistance_cm();  
if(distance > 0 && distance < OBSTACLE_DISTANCE_CM){  
    Robot_Stop();  
    HAL_GPIO_WritePin(LED_OBS_GPIO_Port, LED_OBS_Pin, GPIO_PIN_SET);  
}
```


4. Design detalhado dos blocos do diagrama

Unidade de Controlo:



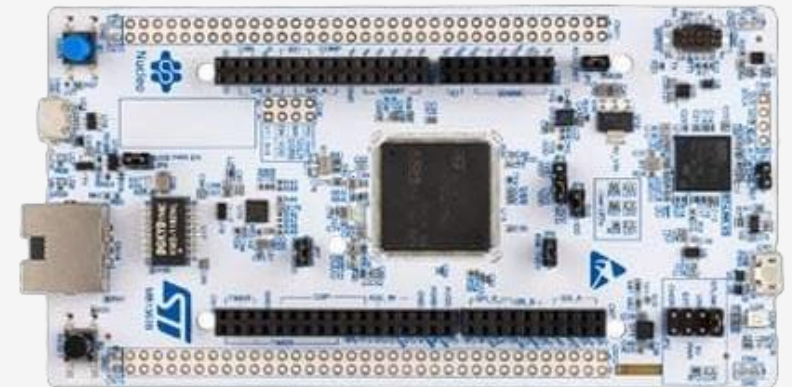
4. Design detalhado dos blocos do diagrama

Unidade de Controlo:

Foi utilizada a placa STM32 NUCLEO-H755ZI-Q como unidade central de controlo do sistema. Esta placa recebe a informação dos sensores, interpreta os valores obtidos e decide a ação a executar.

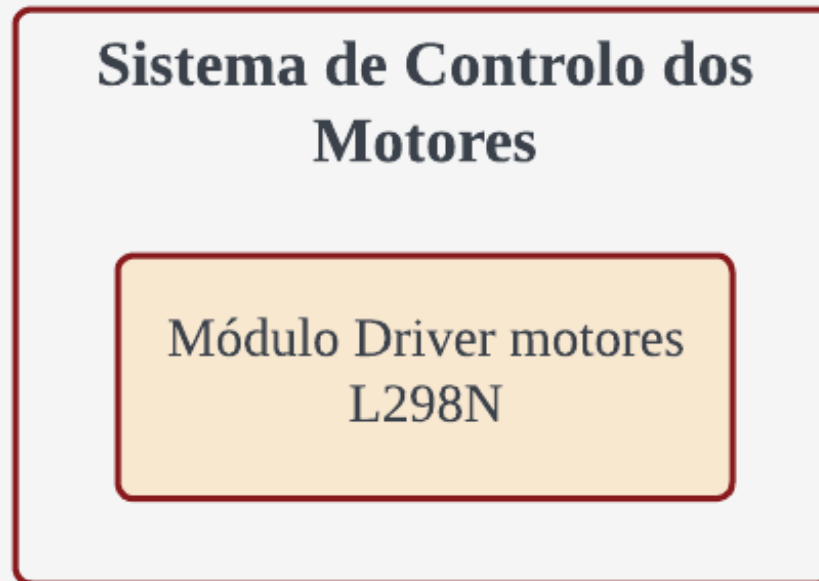
O objetivo deste bloco é controlar o comportamento geral do robô.

A STM32 decide se o robô deve seguir a linha, corrigir a direção, parar perante um obstáculo ou ativar/desativar a sinalização.



4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:



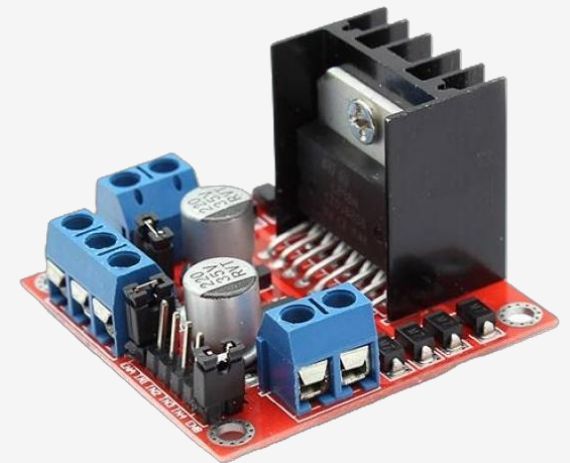
4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Foi utilizado o módulo driver motores L298N para controlar os dois motores DC.

Este módulo recebe sinais da STM32 e permite controlar o sentido de rotação e a velocidade de cada motor.

O objetivo deste bloco é fornecer aos motores a potência necessária para o movimento, permitindo que a STM32 controle os motores de forma segura e independente.



4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Módulo de software: motor_driver.c / motor_driver.h

Hardware associado: Driver L298N + motores DC

Interface:

```
void Motor_Init(void);  
void Motor_SetLeftSpeed(int speed);  
void Motor_SetRightSpeed(int speed);  
void Motor_Stop(void);
```

4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Algoritmo das funções:

Motor_Init()

- Inicializa o controlo dos motores;
- Ativa os dois canais PWM do TIM3;
 - TIM_CHANNEL_1 controla ENA;
 - TIM_CHANNEL_2 controla ENB;
- No final, chama Motor_Stop() para garantir que os motores começam parados.

4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Algoritmo das funções:

Motor_SetLeftSpeed(int speed)

- Recebe a velocidade desejada para o motor esquerdo;
- Limita o valor da velocidade ao intervalo permitido;
- Se a velocidade for positiva:
 - Define o sentido de rotação para a frente;
 - Aplica PWM no canal TIM3_CH2.
- Se a velocidade for zero:
 - Coloca o PWM a zero.

4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Algoritmo das funções:

Motor_SetRightSpeed(int speed)

- Recebe a velocidade desejada para o motor direito;
- Limita o valor da velocidade ao intervalo permitido;
- Se a velocidade for positiva:
 - Define o sentido de rotação para a frente;
 - Aplica PWM no canal TIM3_CH1.
- Se a velocidade for zero:
 - Coloca o PWM a zero.

4. Design detalhado dos blocos do diagrama

Sistema de Controlo dos Motores:

Algoritmo das funções:

Motor_Stop()

- Para os dois motores;
- Coloca os dois sinais PWM a zero;
- Coloca todos os pinos de direção do L298N a nível baixo.

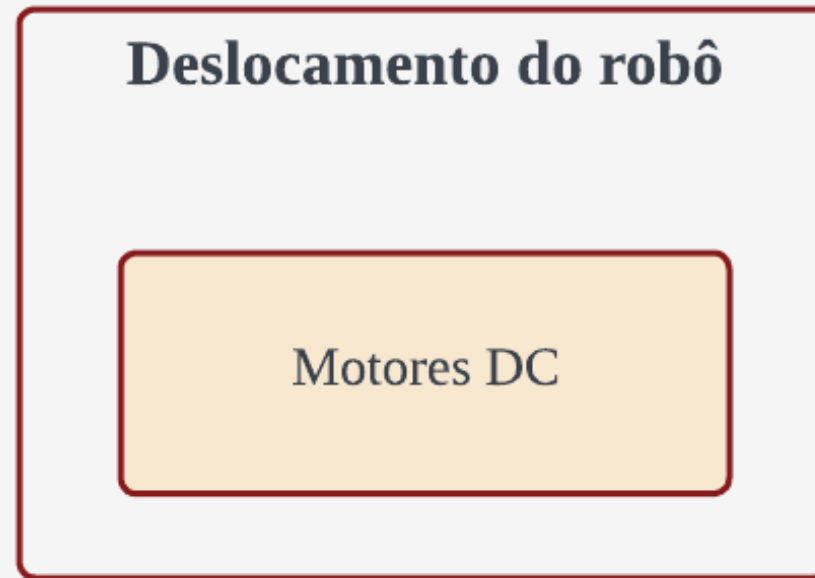
Função auxiliar interna:

LimitSpeed(int speed)

- Garante que o valor de velocidade não ultrapassa o valor máximo permitido pelo PWM;
- O valor é limitado ao intervalo entre -999 e 999.

4. Design detalhado dos blocos do diagrama

Deslocamento do robô:



4. Design detalhado dos blocos do diagrama

Deslocamento do robô:

Foram utilizados dois motores DC para permitir o movimento do robô.

Cada motor é controlado através do driver de motores.

Permite assim que o robô avance, pare ou corrija a direção.

Ao variar a velocidade de cada motor, o robô consegue seguir a linha, e adaptar-se ao percurso.



4. Design detalhado dos blocos do diagrama

Deslocamento do robô:

Módulo de software: robot_motion.c / robot_motion.h

Interface:

void Robot_LineForward(void);

void Robot_LineLeft(void);

void Robot_LineRight(void);

void Robot_Stop(void);

4. Design detalhado dos blocos do diagrama

Deslocamento do robô:

Algoritmo das funções:

Robot_LineForward()

- Define as velocidades dos dois motores para o robô avançar em frente;
- É usada quando o sensor central (IR3) detecta a linha.

Robot_LineLeft()

- Corrige a direção do robô para a esquerda;
- Reduz a velocidade do motor esquerdo;
- Aumenta a velocidade do motor direito;
- É usada quando os sensores da esquerda (IR1 e IR2) detectam a linha.

4. Design detalhado dos blocos do diagrama

Deslocamento do robô:

Algoritmo das funções:

Robot_LineRight()

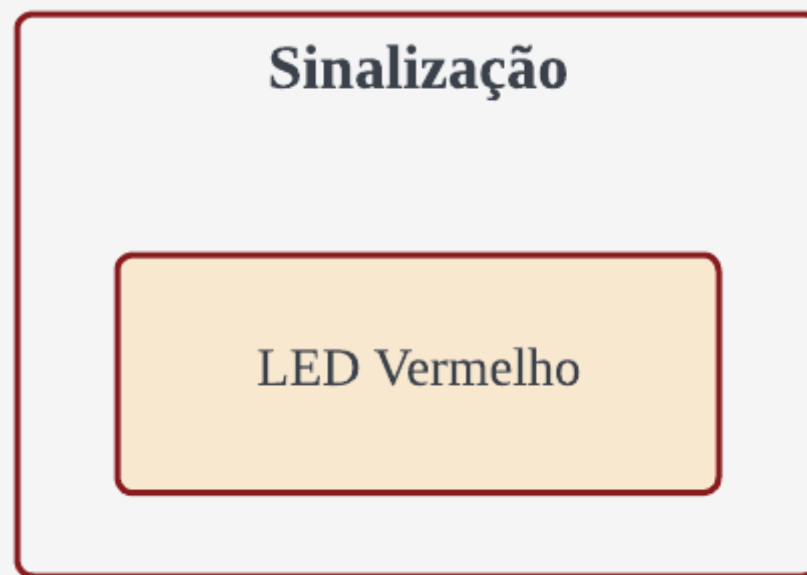
- Corrige a direção do robô para a direita;
- Reduz a velocidade do motor direito;
- Aumenta a velocidade do motor esquerdo;
- É usada quando os sensores da direita (IR4 e IR5) detetam a linha.

Robot_Stop()

- Para o robô através da chamada da função Motor_Stop();
- É usada quando é detetado um obstáculo, ou quando a linha é perdida.

4. Design detalhado dos blocos do diagrama

Sinalização:



4. Design detalhado dos blocos do diagrama

Sinalização:

Foi utilizado um LED vermelho como sinalização da presença de um obstáculo, controlado diretamente pela STM32.

O objetivo deste bloco é indicar quando o robô detecta um obstáculo. Quando o obstáculo é detectado, o LED liga. Quando o obstáculo desaparece, o LED desliga e o robô retoma o funcionamento.



4. Design detalhado dos blocos do diagrama

Sinalização:

Módulo de software: controlo direto na main.c

Interface:

```
HAL_GPIO_WritePin(LED_OBS_GPIO_Port, LED_OBS_Pin, GPIO_PIN_SET);
```

```
HAL_GPIO_WritePin(LED_OBS_GPIO_Port, LED_OBS_Pin, GPIO_PIN_RESET);
```

Algoritmo:

- Verificar se existe obstáculo;
- Se existir obstáculo, ligar o LED vermelho;
- Se não existir obstáculo, desligar o LED vermelho.